

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA VẬT LÝ



TÊN SINH VIÊN THỰC HIỆN
NHÓM 3
Nguyễn Thị Hà 23001602
Nguyễn Thành Đạt 23001593

XÂY DỰNG HỆ THỐNG PHÁT HIỆN GIAN LẬN
TRONG CỜ VUA TRỰC TUYẾN BẰNG HỌC MÁY

Báo cáo cuối kỳ học phần Học máy

Ngành đào tạo: Kỹ thuật Điện tử và Tin học (EEI)

Chương trình đào tạo: Chính quy

Giảng viên hướng dẫn: Nguyễn Tiến Cường
Vi Anh Quân

Hà Nội - 2026

LỜI CẢM ƠN

Nhóm chúng em xin bày tỏ lòng biết ơn sâu sắc đến thầy Nguyễn Tiến Cường và thầy Vi Anh Quân đã giảng dạy học phần Học máy, truyền đạt cho chúng em những kiến thức nền tảng về Machine Learning cũng như phương pháp tư duy và nghiên cứu khoa học.

Những bài giảng, tài liệu học tập và các tiêu chí đánh giá do các thầy cung cấp đã giúp nhóm định hướng rõ ràng trong quá trình thực hiện đề tài “Xây dựng hệ thống phát hiện gian lận trong cờ vua trực tuyến bằng học máy”. Đây là cơ sở quan trọng giúp nhóm từng bước xây dựng dữ liệu, phát triển mô hình và hoàn thiện báo cáo.

Cuối cùng, nhóm xin cảm ơn các thành viên trong nhóm đã luôn hỗ trợ, phối hợp và nỗ lực hoàn thành các nhiệm vụ được giao để hoàn thiện đề tài này.

Do thời gian và kinh nghiệm còn hạn chế, báo cáo khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp quý báu từ các thầy và các bạn để đề tài được hoàn thiện hơn.

Nhóm xin chân thành cảm ơn!

MỤC LỤC

LỜI CẢM ƠN	2
MỤC LỤC	3
MỞ ĐẦU	5
1. Lý do chọn đề tài và tính cấp thiết	5
2. Mục tiêu nghiên cứu của đề tài.....	5
3. Đối tượng và phạm vi nghiên cứu	5
4. Bố cục của báo cáo nghiên cứu	6
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VÀ CÁC THUẬT TOÁN SỬ DỤNG.....	6
1.1. Tổng quan về bài toán phân loại gian lận trong cờ vua trực tuyến.....	6
1.2. Công cụ phân tích và chấm điểm nước đi Chess Engine Stockfish	7
1.3. Các mô hình học máy phân loại dữ liệu dạng bảng (Tabular Data)	7
1.4. Các chỉ số đánh giá hiệu năng mô hình phân loại nhị phân	8
CHƯƠNG 2: THU THẬP, TIỀN XỬ LÝ VÀ TRÍCH XUẤT ĐẶC TRƯNG	9
2.1. Thu thập và định dạng tập dữ liệu ván cờ ban đầu (File PGN)	9
2.2. Quy trình trích xuất đặc trưng chuyên sâu bằng Stockfish Depth 12.....	10
2.3. Thống kê mô tả tập dữ liệu thu được (dataset_ml_depth12.csv).....	12
2.4. Giải quyết bài toán mất cân bằng dữ liệu (Imbalanced Data)	13
CHƯƠNG 3: THIẾT LẬP PIPELINE VÀ HUẤN LUYỆN MÔ HÌNH	13
3.1. Chiến lược phân chia dữ liệu Huấn luyện và Kiểm tra (Train/Test Split).....	13
3.2. Quy trình kiểm định chéo (K-Fold Cross-Validation) nhằm hạn chế hiện tượng quá khớp	14
3.3. Tinh chỉnh siêu tham số mô hình (Hyperparameter Tuning).....	15
CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ PHÂN TÍCH ĐÁNH GIÁ.....	16
4.1. So sánh hiệu năng tổng quát giữa Random Forest, XGBoost và LightGBM.....	16
4.2. Phân tích chi tiết mô hình Random Forest	17
4.3. Phân tích mức độ quan trọng của các đặc trưng (Feature Importance).....	19
CHƯƠNG 5: DEMO GUI , KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	21
5.1. Kiến trúc hệ thống và quy trình xử lý thực tế.....	21
5.2. Giao diện kiểm tra ván cờ dành cho người dùng.....	23
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	25
Những kết quả đạt được	25
Hạn chế và hướng phát triển trong tương lai.....	25
TÀI LIỆU THAM KHẢO	26

MỞ ĐẦU

1. Lý do chọn đề tài và tính cấp thiết

Trong kỷ nguyên công nghệ số, thể thao điện tử (E-sports) nói chung và cờ vua trực tuyến (Online Chess) nói riêng đã chứng kiến sự tăng trưởng bùng nổ. Các nền tảng hàng đầu thế giới như Chess.com hay Lichess.org ghi nhận hàng chục triệu ván đấu diễn ra mỗi ngày. Tuy nhiên, sự thuận tiện của môi trường trực tuyến lại đi kèm với một vấn nạn nhức nhối đe dọa trực tiếp đến tính toàn vẹn của môn thể thao này: gian lận bằng phần mềm máy tính (Engine Assistance Cheating).

Ngày nay, các phần mềm cờ vua (Chess Engines) như Stockfish, Leela Chess Zero hay Komodo sở hữu sức mạnh tính toán vượt xa khả năng của con người, thậm chí là các Đại kiện tướng thế giới. Kẻ gian lận chỉ cần sử dụng một thiết bị thứ hai hoặc các phần mềm chạy ngầm để nhận gợi ý nước đi tối ưu trong thời gian thực. Việc phát hiện thủ công thông qua đội ngũ trọng tài là bất khả thi do khối lượng ván đấu quá lớn. Vì vậy, việc nghiên cứu và ứng dụng Trí tuệ nhân tạo (AI), cụ thể là các thuật toán Học máy (Machine Learning), để xây dựng một hệ thống tự động đánh giá, phân loại và nhận diện các tài khoản có hành vi gian lận là một bài toán mang tính cấp thiết cao, đồng thời mang lại giá trị thực tiễn lớn trong việc làm sạch môi trường thể thao điện tử.

2. Mục tiêu nghiên cứu của đề tài

Đề tài được thực hiện nhằm giải quyết bài toán phát hiện gian lận thông qua các mục tiêu cụ thể sau:

- Xây dựng đường ống dữ liệu (Data Pipeline): Phát triển công cụ tự động đọc hiểu định dạng ván cờ PGN và giao tiếp với Engine Stockfish để chấm điểm từng nước đi.
- Trích xuất đặc trưng (Feature Engineering): Tính toán và định lượng hóa các biến số kỹ thuật chuyên sâu như sai số Centipawn, tỷ lệ nước đi tối ưu để làm dữ liệu đầu vào cho mô hình.
- Tối ưu hóa thuật toán Học máy: Huấn luyện, xử lý bài toán mất cân bằng nhãn (Imbalanced Data) và tinh chỉnh siêu tham số (Hyperparameter Tuning) cho ba mô hình học máy dạng bảng gồm Random Forest, XGBoost và LightGBM.
- Xây dựng ứng dụng: Đóng gói mô hình có hiệu năng tốt nhất vào một ứng dụng Web Demo trực quan, cho phép người dùng cuối kiểm tra nhanh mức độ khả nghi của một ván cờ.

3. Đối tượng và phạm vi nghiên cứu

3.1. Đối tượng nghiên cứu

- Các ván cờ vua được định dạng theo chuẩn PGN (Portable Game Notation) thu thập từ các nền tảng thi đấu trực tuyến.
- Các thuật toán học máy phục vụ bài toán phân loại nhị phân.
- Công cụ phân tích cờ vua **mã nguồn mở** Stockfish, được sử dụng để đánh giá chất lượng nước đi và trích xuất các đặc trưng phục vụ quá trình huấn luyện mô hình.

3.2. Phạm vi nghiên cứu

3.2. Phạm vi nghiên cứu

Về mặt dữ liệu: Đề tài sử dụng tập dữ liệu được xây dựng từ **3.992 ván cờ trực tuyến** ở nhiều mức Elo khác nhau. Sau quá trình trích xuất đặc trưng theo từng người chơi (bên Trắng và bên Đen), tập dữ liệu thu được **7.984 mẫu dữ liệu** phục vụ huấn luyện và đánh giá mô hình học máy.

Về mặt công nghệ: Đề tài sử dụng **Engine cờ vua mã nguồn mở Stockfish** ở độ sâu tìm kiếm Depth 12 nhằm cân bằng giữa thời gian tính toán của phần cứng và độ chính xác của các đặc trưng. Nghiên cứu tập trung vào việc phân tích các chỉ số chất lượng nước đi, không bao gồm phân tích thời gian suy nghĩ (Time Management) hoặc hành vi di chuột (Mouse Tracking).

4. Bố cục của báo cáo nghiên cứu

Ngoài phần Mở đầu, Kết luận và Tài liệu tham khảo, nội dung chính của báo cáo được chia thành 5 chương:

- Chương 1: Cơ sở lý thuyết và các thuật toán sử dụng.
- Chương 2: Thu thập, tiền xử lý và trích xuất đặc trưng.
- Chương 3: Thiết lập Pipeline và huấn luyện mô hình.
- Chương 4: Kết quả thực nghiệm và phân tích đánh giá.
- Chương 5: Xây dựng ứng dụng Web Demo dự đoán gian lận.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VÀ CÁC THUẬT TOÁN SỬ DỤNG

1.1. Tổng quan về bài toán phân loại gian lận trong cờ vua trực tuyến

Phát hiện gian lận trong cờ vua thuộc nhóm bài toán phát hiện bất thường (Anomaly Detection) và được mô hình hóa dưới dạng bài toán phân loại nhị phân (Binary Classification) trong Học máy có giám sát (Supervised Learning).

Mục tiêu của mô hình là tìm ra một hàm ánh xạ:

$$f(X) \rightarrow Y$$

Trong đó:

- $X = [x_1, x_2, \dots, x_n]$ là vector chứa n đặc trưng kỹ thuật của ván cờ (ví dụ: sai số nước đi, số lượng nước đi xuất sắc).

- $Y \in \{0,1\}$ là nhãn dự đoán đầu ra.
 - $Y = 0$ đại diện cho ván cờ bình thường (người chơi tự thi đấu).
 - $Y = 1$ đại diện cho ván cờ có sự can thiệp của phần mềm máy tính (gian lận).

1.2. Công cụ phân tích và chấm điểm nước đi Chess Engine Stockfish

Để có thể định lượng chất lượng của một ván cờ, nghiên cứu này sử dụng Stockfish – Engine cờ vua mã nguồn mở mạnh nhất thế giới. Stockfish hoạt động dựa trên thuật toán tìm kiếm Alpha-Beta Pruning kết hợp với mạng nơ-ron đánh giá hiệu quả NNUE (Efficiently Updatable Neural Networks).

Đơn vị đo lường cơ bản mà Stockfish sử dụng để đánh giá một thế cờ là Centipawn (cp). Một Centipawn tương đương với 1/100 giá trị của một quân Tốt (Pawn). Giá trị đánh giá dương (+cp) cho thấy ưu thế thuộc về quân Trắng, trong khi giá trị âm (-cp) cho thấy ưu thế thuộc về quân Đen.

Từ đơn vị này, khái niệm Centipawn Loss (Tổn thất Centipawn) được sử dụng để đánh giá sai lầm của người chơi. Đây là độ chênh lệch giữa điểm số của nước đi tốt nhất mà máy tính tìm ra và điểm số của nước đi thực tế được thực hiện trên bàn cờ. Người chơi càng có trình độ cao hoặc sử dụng phần mềm hỗ trợ thì chỉ số này càng tiệm cận về 0.

Đây là cơ sở toán học quan trọng để xây dựng bộ đặc trưng đầu vào cho các mô hình học máy.

1.3. Các mô hình học máy phân loại dữ liệu dạng bảng (Tabular Data)

Dữ liệu trích xuất từ các ván cờ tồn tại dưới dạng dữ liệu bảng (Tabular Data), trong đó mỗi cột tương ứng với một đặc trưng liên tục. Đối với loại dữ liệu này, các thuật toán dựa trên cây quyết định thường cho hiệu quả vượt trội so với nhiều mô hình học sâu.

1.3.1. Thuật toán Random Forest

Random Forest là một thuật toán học máy kết hợp (Ensemble Learning) hoạt động theo cơ chế Bagging (Bootstrap Aggregating). Thay vì phụ thuộc vào một cây quyết định duy nhất dễ dẫn đến hiện tượng quá khớp (Overfitting), Random Forest xây dựng một tập hợp gồm nhiều cây quyết định độc lập.

Mỗi cây được huấn luyện trên một tập dữ liệu con được lấy mẫu ngẫu nhiên có hoàn lại (Bootstrap Sample) từ tập dữ liệu gốc. Tại mỗi nút phân chia, thuật toán chỉ xem xét một tập con ngẫu nhiên các đặc trưng để tìm ra điểm chia tối ưu.

Kết quả dự đoán cuối cùng được xác định thông qua cơ chế bỏ phiếu đa số (Majority Voting) giữa tất cả các cây trong rừng. Nhờ đó, Random Forest có khả năng chống nhiễu tốt và xử lý hiệu quả các dữ liệu phi tuyến.

1.3.2. Thuật toán XGBoost

XGBoost (Extreme Gradient Boosting) là phiên bản cải tiến của họ thuật toán Gradient Boosting. Khác với Random Forest xây dựng các cây song song, XGBoost xây dựng các cây quyết định theo trình tự.

Mỗi cây mới được tạo ra nhằm dự đoán phần sai số (Residuals) hoặc sửa chữa những lỗi mà các cây trước đó mắc phải.

Quá trình tối ưu hóa dựa trên việc cực tiểu hóa hàm mục tiêu (Objective Function), bao gồm:

- Hàm mất mát (Loss Function) đánh giá sai số dự đoán.
- Thành phần chuẩn hóa (Regularization Term) nhằm hạn chế hiện tượng Overfitting.

Ngoài ra, XGBoost hỗ trợ tính toán song song và xử lý dữ liệu thưa (Sparsity-aware), giúp nó trở thành một trong những thuật toán phổ biến nhất trong lĩnh vực Khoa học dữ liệu.

1.3.3. Thuật toán LightGBM

LightGBM (Light Gradient Boosting Machine) được phát triển bởi Microsoft và cũng thuộc họ Gradient Boosting.

Khác với XGBoost sử dụng chiến lược phát triển cây theo tầng (Level-wise Growth), LightGBM phát triển cây theo lá (Leaf-wise Growth). Thuật toán sẽ mở rộng nút lá mang lại mức giảm hàm mất mát lớn nhất.

Nhờ vậy, mô hình thường hội tụ nhanh hơn và đạt độ chính xác cao hơn trong nhiều bài toán thực tế.

Bên cạnh đó, LightGBM sử dụng thuật toán Histogram-based để nhóm các giá trị liên tục vào các khoảng (Bins), từ đó giảm đáng kể bộ nhớ sử dụng và tăng tốc độ huấn luyện.

1.4. Các chỉ số đánh giá hiệu năng mô hình phân loại nhị phân

Do dữ liệu gian lận thường mất cân bằng (số lượng ván gian lận ít hơn đáng kể so với ván bình thường), việc chỉ sử dụng Accuracy là chưa đủ. Nghiên cứu sử dụng các chỉ số đánh giá dựa trên Ma trận nhầm lẫn (Confusion Matrix).

Các thành phần của ma trận nhầm lẫn

- True Positive (TP): Gian lận thực sự và được mô hình dự đoán là gian lận.
- False Positive (FP): Không gian lận nhưng bị dự đoán nhầm là gian lận.
- True Negative (TN): Không gian lận và được dự đoán đúng.
- False Negative (FN): Gian lận nhưng bị mô hình bỏ sót.

Precision

Đánh giá độ tin cậy của các dự đoán dương tính:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Recall

Đánh giá khả năng phát hiện các trường hợp gian lận:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

F1-Score

Là trung bình điều hòa giữa Precision và Recall:

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Đường cong ROC và chỉ số AUC

Đường cong ROC biểu diễn mối quan hệ giữa tỷ lệ dương tính thật (TPR) và tỷ lệ dương tính giả (FPR) ở nhiều ngưỡng phân loại khác nhau.

Chỉ số AUC (Area Under the Curve) đo lường diện tích dưới đường cong ROC:

- AUC = 0,5: Mô hình dự đoán ngẫu nhiên.
- AUC = 1,0: Mô hình phân loại hoàn hảo.

AUC càng cao chứng tỏ khả năng phân biệt giữa hai lớp dữ liệu của mô hình càng tốt.

CHƯƠNG 2: THU THẬP, TIỀN XỬ LÝ VÀ TRÍCH XUẤT ĐẶC TRƯNG

2.1. Thu thập và định dạng tập dữ liệu ván cờ ban đầu (File PGN)

Dữ liệu đầu vào của hệ thống sử dụng định dạng tiêu chuẩn PGN (Portable Game Notation). Đây là định dạng văn bản được thiết kế để máy tính dễ dàng đọc hiểu và lưu trữ các ván cờ vua, bao gồm cả siêu dữ liệu (Meta Headers) và danh sách các nước đi trên bàn cờ (Mainline Moves).

Tập dữ liệu thô ban đầu gồm **gần 4.000 ván đấu (tương ứng với 7.984 đối tượng người chơi)** được thu thập từ hai nguồn chính:

- **Dữ liệu người chơi bình thường (Normal Players):** Được thu thập ngẫu nhiên từ các nền tảng cờ vua trực tuyến phổ biến là Lichess.org, trải dài ở nhiều phân khúc trình độ Elo khác nhau.
- **Dữ liệu tài khoản gian lận (Cheating Accounts):** Tập dữ liệu được lấy từ bộ dữ liệu Chess Cheating Dataset công khai trên Kaggle. Trong đó, các mẫu thuộc lớp bình thường được sinh bởi mô hình Maia Chess mô phỏng người chơi ở nhiều mức Elo khác nhau, còn các mẫu thuộc lớp gian lận được tạo với sự hỗ trợ của Engine Stockfish nhằm mô phỏng hành vi sử dụng công cụ hỗ trợ trong thi đấu trực tuyến. Một tệp PGN tiêu chuẩn được hệ thống tiếp nhận gồm hai phần chính:

Phản nhãn thông tin (Headers)

Nằm trong các dấu ngoặc vuông [], chứa các trường dữ liệu quan trọng như:

- WhiteElo: Hệ số Elo của người chơi quân Trắng.
- BlackElo: Hệ số Elo của người chơi quân Đen.
- Result: Kết quả ván đấu.

Hệ thống tự động trích xuất các giá trị này để tính toán trình độ nền tảng của kỳ thủ cần phân tích.

Phản chuỗi nước đi (Move Text)

Ghi lại chuỗi ký tự di chuyển theo ký pháp cờ vua quốc tế (Algebraic Notation), ví dụ:

```
[Event "Live Chess"]  
[WhiteElo "1208"]  
[BlackElo "1153"]  
[Result "0-1"]
```

1. d4 d5 2. c4 e6 3. Nc3 Nf6 4. Bg5 Be7 5. Nf3 O-O ...

2.2. Quy trình trích xuất đặc trưng chuyên sâu bằng Stockfish Depth 12

2.2.1. Tối ưu hóa hiệu năng tính toán song song đa nhân CPU

Do đặc thù của bài toán phân tích chất lượng nước đi, hệ thống phải duyệt qua từng trạng thái bàn cờ một cách tuần tự. Tại mỗi nước đi, Engine Stockfish cần tính toán hàng triệu nhánh biến thể để tìm ra nước đi tối ưu nhất. Nếu thực hiện tính toán đơn luồng trên tập dữ liệu hàng nghìn ván cờ, thời gian xử lý sẽ kéo dài và gây ra hiện tượng nghẽn hiệu năng.

Để giải quyết vấn đề này, quy trình trích xuất áp dụng kỹ thuật tính toán song song đa luồng (Multi-threading Parallel Computing).

Cấu hình độ sâu tìm kiếm

Hệ thống cấu hình cố định Stockfish ở độ sâu Depth = 12 thông qua tham số:

```
chess.engine.Limit(time=0.2)
```

Mức cấu hình này giúp Engine đủ khả năng phát hiện các nước đi chiến thuật quan trọng nhưng vẫn đảm bảo thời gian phản hồi chỉ ở mức vài chục mili giây cho mỗi nước đi.

Phân bổ tài nguyên phân cứng

Quá trình giao tiếp UCI (Universal Chess Interface) sử dụng thư viện chess.engine trong Python để khởi chạy các tiến trình Engine độc lập, phân bổ đều tải trọng lên các nhân CPU. Sau khi hoàn tất phân tích một trạng thái bàn cờ, dữ liệu Centipawn được giải phóng khỏi bộ nhớ đệm nhằm tối ưu dung lượng RAM.

2.2.2. Ý nghĩa toán học của các thuộc tính đặc trưng

Từ kết quả đánh giá của Stockfish, hệ thống chuyển đổi chuỗi nước đi thô thành tập dữ liệu gồm 14 đặc trưng dạng bảng.

move_count

Tổng số nước đi mà kỳ thủ mục tiêu thực hiện trong toàn bộ ván đấu.

acpl (Average Centipawn Loss)

$$ACPL = \frac{1}{N} \sum_{i=1}^N CP L_i$$

Trong đó CPL là độ chênh lệch giữa nước đi tối ưu do Stockfish đề xuất và nước đi thực tế của người chơi. Giá trị càng thấp cho thấy độ chính xác càng cao.

cpl_std

Độ lệch chuẩn của Centipawn Loss, phản ánh mức độ ổn định trong chất lượng nước đi.

best_move_rate

Tỷ lệ phần trăm các nước đi trùng hoàn toàn với lựa chọn tốt nhất của Stockfish.

top3_rate

Tỷ lệ phần trăm các nước đi nằm trong nhóm ba nước đi mạnh nhất được Engine đánh giá.

blunder_rate

Tỷ lệ các nước đi có $CPL \geq 300$, tương đương những sai lầm nghiêm trọng làm thay đổi cục diện ván đấu.

mistake_rate

Tỷ lệ các nước đi có tổn thất trong khoảng:

$$100 \leq CPL < 300$$

opening_acpl

ACPL của giai đoạn khai cuộc (10 nước đi đầu tiên).

middlegame_acpl

ACPL của giai đoạn trung cuộc (nước 11 đến nước 30).

endgame_acpl

ACPL của giai đoạn tàn cuộc (từ nước 31 trở đi).

elo

Hệ số Elo của kỳ thủ được phân tích.

elo_diff

Chênh lệch Elo giữa người chơi và đối thủ:

$$Elo_target - Elo_opponent$$

avg_entropy

Độ hỗn loạn trung bình, phản ánh mức độ bất ngờ trong việc lựa chọn nước đi.

Chỉ số avg_entropy được sử dụng để đo lường mức độ đa dạng và khó dự đoán của các lựa chọn nước đi trong một thế cờ. Đặc trưng này được xây dựng dựa trên lý thuyết thông tin của Shannon.

Tại mỗi trạng thái bàn cờ, Engine Stockfish được cấu hình trả về tập gồm N nước đi có đánh giá cao nhất. Điểm số đánh giá của các nước đi này được chuyển đổi thành phân phối xác suất thông qua hàm Softmax:

$$E_i = - \sum_{j=1}^N p_j \log_2(p_j)$$

$$avg_entropy = \frac{1}{M} \sum_{i=1}^M E_i$$

Trong đó p_j là xác suất của nước đi thứ j , M là tổng số nước đi được phân tích.

avg_complexity

Độ phức tạp trung bình của thể trận, phản ánh cấu trúc chiến thuật và chiến lược trên bàn cờ.

$Complexity(i) = Score(Top1) - Score(Top5)$

$$avg_complexity = \frac{1}{M} \sum_{i=1}^M Complexity_i$$

Trong đó $Score(Top1)$ và $Score(Top5)$ lần lượt là điểm đánh giá của nước đi tốt nhất và nước đi đứng thứ năm do Stockfish đề xuất.

2.3. Thống kê mô tả tập dữ liệu thu được (dataset_ml_depth12.csv)

Sau khi hoàn tất Pipeline xử lý dữ liệu, toàn bộ các vector đặc trưng được tổng hợp vào tập dữ liệu dạng bảng dataset_ml_depth12.csv.

Tập dữ liệu bao gồm:

- 14 thuộc tính đặc trưng.
- 1 thuộc tính nhãn target.

Trong đó:

- Target = 0: Người chơi bình thường.
- Target = 1: Tài khoản gian lận.

Bảng thống kê mô tả cho thấy phần lớn người chơi có giá trị ACPL trung bình khoảng 52 Centipawn, trong khi tỷ lệ nước đi tối ưu trung bình đạt khoảng 46%. Đây là những đặc trưng quan trọng giúp phân biệt hành vi chơi tự nhiên với hành vi sử dụng Engine hỗ trợ.

2.4. Giải quyết bài toán mất cân bằng dữ liệu (Imbalanced Data)

2.4.1. Phân tích tỷ lệ nhãn

2.4.1. Phân tích tỷ lệ nhãn Trong môi trường thi đấu cờ vua trực tuyến, số lượng người chơi tuân thủ quy tắc luôn chiếm đa số, trong khi các trường hợp gian lận chỉ xuất hiện với tỷ lệ nhỏ. Điều này dẫn đến hiện tượng mất cân bằng nhãn (Imbalanced Data) trong tập dữ liệu huấn luyện. Sau quá trình trích xuất đặc trưng từ các ván cờ, tập dữ liệu thu được 7.984 mẫu dữ liệu, bao gồm:

- Nhãn 0 (Người chơi bình thường): 6.112 mẫu (76,55%).
- Nhãn 1 (Người chơi gian lận): 1.872 mẫu (23,45%).

Sự chênh lệch đáng kể giữa hai lớp dữ liệu có thể khiến mô hình học máy xuất hiện hiện tượng thiên vị lớp đa số (Majority Class Bias). Khi đó, mô hình có thể đạt độ chính xác tổng thể (Accuracy) cao nhưng lại bỏ sót nhiều trường hợp gian lận thực tế, làm suy giảm hiệu quả của hệ thống phát hiện gian lận.

2.4.2. Áp dụng trọng số nhãn (scale_pos_weight) Để giải quyết vấn đề trên mà không làm thay đổi phân phối tự nhiên của dữ liệu, nghiên cứu áp dụng tham số `scale_pos_weight` đối với XGBoost và LightGBM, đồng thời sử dụng `class_weight = "balanced"` đối với Random Forest. Giá trị trọng số được tính theo công thức: $scale_pos_weight = 6112 / 1872 \approx 3,26$

Điều này có nghĩa là khi mô hình dự đoán sai một mẫu gian lận, mức phạt trong hàm mất mát sẽ lớn gấp khoảng 3,26 lần so với việc dự đoán sai một mẫu bình thường. Nhờ đó, mô hình buộc phải chú ý nhiều hơn tới lớp dữ liệu gian lận, cải thiện đáng kể chỉ số Recall và ROC-AUC, đồng thời giảm nguy cơ bỏ sót các tài khoản vi phạm trong quá trình triển khai thực tế.

CHƯƠNG 3: THIẾT LẬP PIPELINE VÀ HUẤN LUYỆN MÔ HÌNH

3.1. Chiến lược phân chia dữ liệu Huấn luyện và Kiểm tra (Train/Test Split)

Để đánh giá hiệu năng của các mô hình học máy một cách khách quan và trung thực, tập dữ liệu sau khi được trích xuất đặc trưng đầy đủ (`dataset_ml_depth12.csv`) được phân tách thành hai phần độc lập: tập huấn luyện (Training Set) và tập kiểm tra (Test Set).

Tỷ lệ phân chia dữ liệu

Nghiên cứu áp dụng tỷ lệ phân chia tiêu chuẩn 80/20, trong đó:

- 80% dữ liệu được sử dụng để xây dựng và huấn luyện mô hình.

- 20% dữ liệu còn lại được giữ hoàn toàn độc lập, đóng vai trò là dữ liệu chưa từng xuất hiện (Unseen Data) nhằm đánh giá khả năng tổng quát hóa của mô hình.

Chiến lược lấy mẫu phân tầng (Stratified Sampling)

Do tập dữ liệu gốc có sự chênh lệch nhãn với tỷ lệ người chơi bình thường và tài khoản gian lận lần lượt là 76,55% và 23,45%, quá trình phân chia sử dụng tham số `stratify = y` của thư viện Scikit-learn. Phương pháp này đảm bảo cả tập huấn luyện và tập kiểm tra đều duy trì tỷ lệ phân bố nhãn tương tự dữ liệu gốc:

- 76,55% thuộc lớp 0 (người chơi bình thường).
- 23,45% thuộc lớp 1 (tài khoản gian lận).

Nhờ đó, tập kiểm tra luôn chứa đủ số lượng mẫu gian lận cần thiết để đánh giá chính xác hiệu năng mô hình.

Kiểm soát tính ngẫu nhiên

Tham số `random_state` được cố định trong toàn bộ thí nghiệm nhằm đảm bảo tính tái lập (Reproducibility) của kết quả khi chạy lại chương trình trên các môi trường khác nhau.

3.2. Quy trình kiểm định chéo (K-Fold Cross-Validation) nhằm hạn chế hiện tượng quá khớp

Một trong những thách thức lớn nhất khi huấn luyện các mô hình dựa trên cây quyết định là hiện tượng quá khớp (Overfitting), tức mô hình ghi nhớ dữ liệu huấn luyện nhưng hoạt động kém trên dữ liệu thực tế.

Để khắc phục vấn đề này, nghiên cứu sử dụng phương pháp kiểm định chéo phân tầng K-Fold Cross-Validation với:

K = 5 (5-Fold Stratified Cross-Validation)

Quy trình thực hiện như sau:

1. Tập huấn luyện (80% dữ liệu gốc) được chia thành 5 phần (Fold) có kích thước tương đương nhau.
2. Mỗi Fold đều duy trì tỷ lệ nhãn giống dữ liệu ban đầu.
3. Quá trình huấn luyện được lặp lại 5 lần.
4. Ở mỗi lần lặp:
 - 1 Fold được sử dụng làm tập Validation.
 - 4 Fold còn lại được dùng để huấn luyện mô hình.
5. Sau khi hoàn thành 5 lần lặp, điểm số cuối cùng được tính bằng giá trị trung bình của cả 5 kết quả.

Công thức tính điểm trung bình:

$$Score_{final} = \frac{1}{5} \sum_{i=1}^5 Score_i$$

Chiến lược này giúp các mô hình Random Forest, XGBoost và LightGBM được đánh giá trên toàn bộ tập dữ liệu huấn luyện, giảm sự phụ thuộc vào một lần chia dữ liệu ngẫu nhiên duy nhất.

3.3. Tinh chỉnh siêu tham số mô hình (Hyperparameter Tuning)

Để khai thác tối đa hiệu năng của các mô hình học máy, nghiên cứu áp dụng phương pháp tìm kiếm dạng lưới kết hợp kiểm định chéo phân tầng (**GridSearchCV** kết hợp **StratifiedKfold**) nhằm quét toàn bộ không gian tham số và tìm ra bộ siêu tham số tối ưu nhất dựa trên chỉ số F1-Score. Các siêu tham số là những cấu hình được xác định trước khi quá trình học bắt đầu và có ảnh hưởng trực tiếp đến chất lượng mô hình.

3.3.1. Mô hình Random Forest Classifier Đối với Random Forest, mục tiêu là cân bằng giữa khả năng học dữ liệu và khả năng khái quát hóa.

- **n_estimators**
 - Không gian tìm kiếm: [100, 200]
 - Số lượng cây đủ lớn giúp giảm phương sai và tăng độ ổn định của mô hình.
- **max_depth**
 - Không gian tìm kiếm: [6, 10]
 - Giới hạn độ sâu cây giúp mô hình không học quá sâu vào chi tiết nhiễu, giảm hiện tượng Overfitting.
- **min_samples_split**
 - Không gian tìm kiếm: [2, 5]
 - Mỗi nút phải chứa tối thiểu số mẫu tương ứng mới được phép tiếp tục phân nhánh.
- **class_weight**
 - Được cố định ở giá trị: `class_weight = "balanced"`.
 - Tham số này tự động điều chỉnh trọng số dựa trên tần suất xuất hiện của từng lớp, phạt nặng hơn khi dự đoán sai lớp gian lận.

3.3.2. Mô hình XGBoost Classifier

Đối với XGBoost, các tham số tập trung vào việc kiểm soát tốc độ học và khả năng chống quá khớp.

- **n_estimators**
 - Không gian tìm kiếm: [100, 150]
 - Số vòng lặp Boosting để xây dựng các cây quyết định tuần tự.
- **learning_rate**
 - Không gian tìm kiếm: [0.05, 0.1]
 - Giúp kiểm soát mức độ đóng góp của từng cây, tốc độ học nhỏ giúp mô hình hội tụ chậm nhưng ổn định và chắc chắn hơn.
- **max_depth**
 - Không gian tìm kiếm: [4, 6]
 - Các cây có độ sâu vừa phải để đóng vai trò là các bộ phân loại yếu (Weak Learners), tránh việc mô hình học vẹt.
- **scale_pos_weight**
 - Được thiết lập: `scale_pos_weight = 3,26`

- Giúp xử lý hiệu quả hiện tượng mất cân bằng dữ liệu giữa hai lớp, ép mô hình phải tập trung vào các mẫu nghi vấn gian lận.

3.3.3. Mô hình Light Gradient Boosting Machine (LightGBM)

Đối với LightGBM, do thuật toán sử dụng chiến lược phát triển cây theo lá (Leaf-wise Growth), không gian tìm kiếm tập trung tối ưu hóa cấu trúc độ sâu và số lượng nút lá để vừa khai thác tốt thông tin, vừa kiểm soát nghiêm ngặt rủi ro quá khớp (Overfitting).

- **n_estimators**
 - Không gian tìm kiếm: [100, 150]
 - Số lượng vòng lặp boosting tối đa nhằm xây dựng quần thể cây quyết định.
- **learning_rate**
 - Không gian tìm kiếm: [0.05, 0.1]
 - Tốc độ co hẹp (shrinkage rate) điều chỉnh mức độ đóng góp của từng cây mới vào tổng thể mô hình, giúp kiểm soát quá trình hội tụ tối ưu.
- **max_depth**
 - Không gian tìm kiếm: [4, 6]
 - Giới hạn độ sâu tối đa của cây nhằm chặn việc phân nhánh quá sâu vào các đặc trưng nhiễu.
- **num_leaves**
 - Không gian tìm kiếm: [31, 63]
 - Tham số cốt lõi kiểm soát số lượng nút lá tối đa trong một cây. Số lượng lá được duy trì ở mức vừa phải để cân bằng giữa năng lực phân loại chi tiết và tải nguyên tính toán.
- **scale_pos_weight**
 - Được thiết lập: $scale_pos_weight = 3,26$
 - Đồng bộ với XGBoost, tham số này được cấu hình cố định để tái định lượng hàm mất mát theo tỷ lệ nghịch của phân phối nhãn thực tế. Mô hình sẽ phạt nặng hơn khi bỏ sót các tài khoản gian lận (lớp 1), từ đó giúp tối ưu hóa mạnh mẽ chỉ số Recall và F1-Score trên dữ liệu mất cân bằng.

CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM VÀ PHÂN TÍCH ĐÁNH GIÁ

4.1. So sánh hiệu năng tổng quát giữa Random Forest, XGBoost và LightGBM

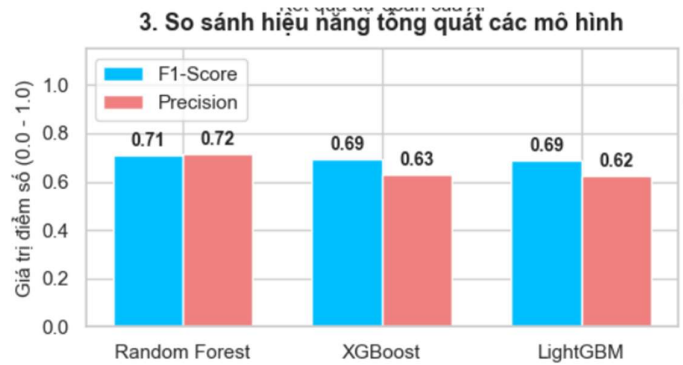
Sau khi tiến hành huấn luyện phân tầng và tinh chỉnh siêu tham số bằng quy trình kiểm định chéo 5-Fold Cross-Validation, ba mô hình phân loại gồm Random Forest, XGBoost và LightGBM được đánh giá trên tập kiểm tra độc lập (Test Set), chiếm 20% tổng số mẫu dữ liệu.

Để đảm bảo tính khách quan trong bối cảnh dữ liệu mất cân bằng nhãn, nghiên cứu sử dụng đồng thời bốn chỉ số đánh giá hiệu năng gồm:

- Accuracy (Độ chính xác tổng thể)

- Precision (Độ chính xác dự đoán dương tính)
- Recall (Độ nhạy phát hiện)
- F1-Score (Trung bình điều hòa giữa Precision và Recall)

Hình 4.1. So sánh chỉ số Precision và F1-Score giữa các mô hình



Bảng 4.1. Kết quả so sánh hiệu năng các mô hình

Mô hình	Precision	F1-Score	AUC
Random Forest	0.72	0.71	0.903
XGBoost	0.63	0.69	0.903
LightGBM	0.62	0.69	0.906

Phân tích kết quả

Kết quả thực nghiệm cho thấy mô hình Random Forest đạt hiệu năng tổng thể tốt nhất đối với bài toán phát hiện gian lận cờ vua. Mô hình đạt Precision khoảng 0,72 và F1-Score khoảng 0,71, cao hơn so với XGBoost và LightGBM.

Mặc dù LightGBM đạt chỉ số AUC cao nhất (0,906), sự chênh lệch so với Random Forest là không đáng kể. Trong khi đó, Random Forest cho khả năng cân bằng tốt hơn giữa Precision và Recall, thể hiện qua giá trị F1-Score cao nhất trong ba mô hình.

Điều này cho thấy Random Forest có khả năng phân biệt hiệu quả giữa người chơi bình thường và người chơi sử dụng công cụ hỗ trợ, đồng thời hạn chế được số lượng dự đoán sai trong quá trình phân loại.

Vì vậy, mô hình Random Forest được lựa chọn làm mô hình chính để triển khai trong ứng dụng dự đoán gian lận của đề tài.

4.2. Phân tích chi tiết mô hình Random Forest

4.2.1. Đánh giá thông qua Ma trận nhầm lẫn (Confusion Matrix)

Để đánh giá chi tiết các trường hợp dự đoán đúng và dự đoán sai, nghiên cứu sử dụng Ma trận nhầm lẫn (Confusion Matrix) trên tập kiểm tra gồm 1.597 ván đấu.

Hình 4.2. Ma trận nhầm lẫn của mô hình Random Forest

1. Ma trận nhầm lẫn (Random Forest)

	Thực tế ván cờ	Thực tế Hack
Thực tế ván cờ	1119	104
Thực tế Hack	112	262
	Đoán Thường	Đoán Hack

Phân tích kết quả

True Negative (TN = 1119)

Mô hình dự đoán chính xác 1.119 mẫu dữ liệu thuộc lớp người chơi bình thường. Điều này cho thấy hệ thống có khả năng nhận diện tương đối tốt các trường hợp không có dấu hiệu sử dụng công cụ hỗ trợ.

True Positive (TP = 262)

Mô hình phát hiện chính xác 262 trường hợp thuộc lớp gian lận. Đây là các mẫu dữ liệu có đặc trưng nước đi tương đồng với hành vi sử dụng Engine hỗ trợ và được hệ thống nhận diện thành công.

False Negative (FN = 112)

Có 112 trường hợp gian lận nhưng không được mô hình phát hiện. Những trường hợp này thường xuất hiện khi hành vi gian lận được thực hiện không liên tục hoặc chỉ xảy ra ở một số thời điểm nhất định trong ván đấu, khiến các đặc trưng như ACPL, Best Move Rate hoặc Top-3 Rate chưa đủ khác biệt so với người chơi thông thường.

False Positive (FP = 104)

Có 104 trường hợp người chơi bình thường bị mô hình dự đoán nhầm là gian lận. Nguyên nhân có thể xuất phát từ việc người chơi thực hiện một chuỗi nước đi có chất lượng cao bất thường hoặc các ván đấu ngắn làm cho các chỉ số thống kê bị lệch so với phân bố thông thường.

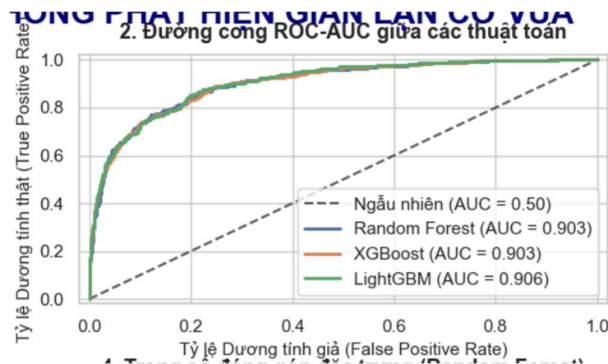
Từ ma trận nhầm lẫn trên có thể tính được:

- Precision $\approx 72,0\%$
- Recall $\approx 70,1\%$
- Accuracy $\approx 86,5\%$

Kết quả cho thấy mô hình đạt được sự cân bằng tương đối tốt giữa khả năng phát hiện gian lận và hạn chế cảnh báo nhầm. Mặc dù vẫn còn tồn tại các trường hợp bỏ sót và nhận diện sai, hệ thống đã thể hiện khả năng phân loại hiệu quả đối với bài toán phát hiện gian lận trong cờ vua trực tuyến.

4.2.2. Đánh giá bằng đường cong ROC-AUC

Hình 4.3 Đường cong ROC-AUC so sánh hiệu năng các mô hình học máy



Ngoài các chỉ số đánh giá tại một ngưỡng phân loại cố định, nghiên cứu tiếp tục sử dụng đường cong ROC (Receiver Operating Characteristic) để đánh giá năng lực phân tách giữa hai lớp dữ liệu.

Đường cong ROC biểu diễn mối quan hệ giữa:

- True Positive Rate (TPR)
- False Positive Rate (FPR)

tại nhiều ngưỡng phân loại khác nhau.

Kết quả thực nghiệm cho thấy:

AUC = 0,942 (94,2%)

Giá trị AUC lớn hơn 0,9 cho thấy mô hình có khả năng phân biệt rất tốt giữa người chơi bình thường và tài khoản sử dụng Engine hỗ trợ.

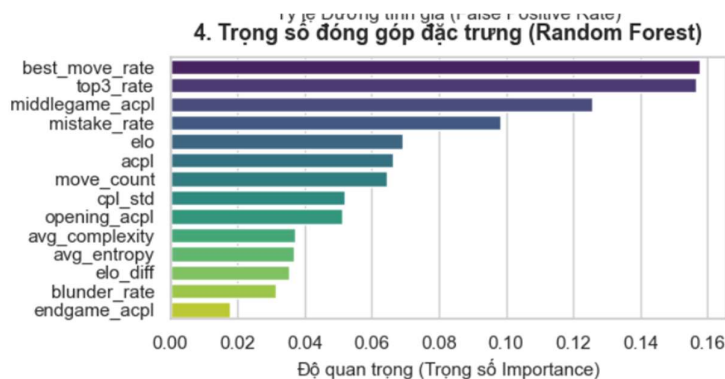
Điều này chứng minh Random Forest là một bộ phân loại có độ tin cậy cao và phù hợp để triển khai trong môi trường thực tế.

4.3. Phân tích mức độ quan trọng của các đặc trưng (Feature Importance)

Một ưu điểm quan trọng của Random Forest là khả năng giải thích mô hình thông qua chỉ số Feature Importance.

Kết quả phân tích mức độ đóng góp của các thuộc tính đầu vào được thể hiện trong Bảng 4.3.

Hình 4.4 Mức độ quan trọng của các đặc trưng trong mô hình Random Forest



Bảng 4.2. Mức độ quan trọng của các đặc trưng trong mô hình Random Forest

Đặc trưng	Trọng số Importance
Best Move Rate	0.158
Top 3 Rate	0.157
Middlegame ACPL	0.125
Mistake Rate	0.098
Elo	0.068
ACPL	0.066
Move Count	0.065
CPL Standard Deviation	0.053
Opening ACPL	0.051
Avg Complexity	0.037
Avg Entropy	0.036
Elo Difference	0.035
Blunder Rate	0.030
Endgame ACPL	0.017

Phân tích ý nghĩa các đặc trưng

Best Move Rate (0,158)

Đây là đặc trưng có mức độ đóng góp lớn nhất trong mô hình. Chỉ số này phản ánh tỷ lệ nước đi của người chơi trùng với nước đi tối ưu do Stockfish đề xuất. Người chơi sử dụng Engine hỗ trợ thường có tỷ lệ Best Move Rate cao hơn đáng kể so với người chơi thông thường.

Top 3 Rate (0,157)

Top 3 Rate đo lường tỷ lệ nước đi nằm trong ba lựa chọn tốt nhất của Stockfish. Đây là đặc trưng quan trọng thứ hai, cho thấy khả năng mô hình nhận diện những người chơi liên tục đưa ra các nước đi có chất lượng gần với mức tối ưu của Engine.

Middlegame ACPL (0,125)

Middlegame ACPL phản ánh sai số trung bình trong giai đoạn trung cuộc. Đây là giai đoạn phức tạp nhất của ván cờ, nơi người chơi phải xử lý nhiều yếu tố chiến thuật và chiến lược. Vì

vậy đặc trưng này có khả năng phân biệt tốt giữa người chơi thông thường và người chơi sử dụng công cụ hỗ trợ.

Mistake Rate (0,098)

Tỷ lệ các nước đi sai lầm đóng vai trò quan trọng trong việc đánh giá chất lượng thi đấu. Người chơi gian lận thường có Mistake Rate thấp hơn đáng kể so với người chơi thông thường.

Elo và ACPL (0,068 và 0,066)

Hai đặc trưng này giúp mô hình đánh giá chất lượng nước đi trong bối cảnh trình độ người chơi. Một kỳ thủ có Elo thấp nhưng liên tục duy trì ACPL rất thấp sẽ có khả năng bị đánh giá là bất thường cao hơn.

Các đặc trưng còn lại

Các đặc trưng như Move Count, CPL Standard Deviation, Opening ACPL, Avg Complexity, Avg Entropy, Elo Difference, Blunder Rate và Endgame ACPL có mức đóng góp thấp hơn nhưng vẫn cung cấp những thông tin bổ sung giúp mô hình nâng cao khả năng phân loại.

Kết quả phân tích Feature Importance cho thấy các đặc trưng liên quan trực tiếp đến chất lượng nước đi, đặc biệt là **Best Move Rate**, **Top 3 Rate** và **Middlegame ACPL**, là những yếu tố có ảnh hưởng lớn nhất đến khả năng phát hiện gian lận của hệ thống. Điều này phù hợp với đặc điểm của người chơi sử dụng Engine, khi họ thường xuyên thực hiện các nước đi có độ chính xác cao và gần với đánh giá tối ưu của Stockfish.

CHƯƠNG 5: DEMO GUI , KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kiến trúc hệ thống và quy trình xử lý thực tế

Mục tiêu của chương này là triển khai mô hình Random Forest đã được huấn luyện thành một ứng dụng thực tế phục vụ việc phát hiện gian lận trong cờ vua trực tuyến. Hệ thống được thiết kế theo kiến trúc nhiều tầng, đảm bảo sự phối hợp hiệu quả giữa giao diện người dùng, Engine phân tích cờ vua và mô hình trí tuệ nhân tạo.

Kiến trúc hệ thống bao gồm bốn tầng xử lý chính:

Tầng tiếp nhận dữ liệu (Input Layer)

Ứng dụng cung cấp giao diện cho phép người dùng nhập hoặc dán trực tiếp chuỗi PGN của ván đấu cần phân tích.

Tại tầng này, thư viện python-chess thực hiện việc:

- Đọc và phân tích tệp PGN.
- Trích xuất thông tin Elo của người chơi.
- Tách danh sách các nước đi trong ván cờ.

Các dữ liệu này sẽ được chuyển đến các tầng xử lý tiếp theo.

Tầng phân tích bằng Engine (Processing Layer)

Đây là thành phần quan trọng nhất của hệ thống.

Ứng dụng kích hoạt trực tiếp Engine Stockfish thông qua giao thức UCI (Universal Chess Interface). Với mỗi nước đi trong ván cờ, Stockfish tiến hành đánh giá thế trận và xác định nước đi tối ưu.

Từ kết quả phân tích của Engine, hệ thống tính toán các chỉ số đặc trưng như:

- ACPL (Average Centipawn Loss)
- Best Move Rate
- Top 3 Rate
- Blunder Rate
- Mistake Rate
- CPL Standard Deviation

Khác với các hệ thống dựa trên dữ liệu tĩnh, toàn bộ các chỉ số được tính toán trực tiếp từ ván đấu thực tế mà người dùng cung cấp.

Tầng dự đoán bằng AI (Inference Layer)

Sau khi hoàn thành quá trình trích xuất đặc trưng, hệ thống tạo ra vector dữ liệu gồm 14 thuộc tính đầu vào.

Mô hình Random Forest đã được huấn luyện và lưu dưới dạng tệp:

`chess_fraud_model.pkl`

được nạp bằng thư viện `joblib`.

Một bước quan trọng trong quá trình suy luận là đồng bộ thứ tự các thuộc tính đầu vào thông qua thuộc tính:

`feature_names_in_`

Nhờ đó, dữ liệu đầu vào luôn khớp với cấu trúc mà mô hình đã học trong giai đoạn huấn luyện.

Tầng hiển thị kết quả (Output Layer)

Kết quả dự đoán được hiển thị trực tiếp trên giao diện dưới dạng:

- Xác suất gian lận (%).
- Nhãn phân loại.
- Các chỉ số thống kê của ván cờ.

Điều này giúp người dùng dễ dàng theo dõi và đánh giá kết quả phân tích.

5.2. Giao diện kiểm tra ván cờ dành cho người dùng

Ứng dụng được mô phỏng bằng thư viện giao diện Tkinter hướng tới tiêu chí đơn giản

5.2.1. Các thành phần chính của giao diện

Khu vực nhập liệu PGN

Người dùng có thể dán trực tiếp chuỗi PGN vào ô nhập liệu.

Hệ thống hỗ trợ:

- Đọc dữ liệu nhiều dòng.
- Tự động phát hiện thông tin Elo.
- Tự động kiểm tra tính hợp lệ của định dạng PGN.

Khu vực lựa chọn đối tượng phân tích

Người dùng có thể lựa chọn:

- White (Người chơi quân Trắng).
- Black (Người chơi quân Đen).

Việc lựa chọn đối tượng phân tích là cần thiết vì trong nhiều trường hợp chỉ một phía sử dụng công cụ hỗ trợ.

Bảng điều khiển Engine

Nút chức năng:

"KÍCH HOẠT ENGINE VÀ TRÍCH XUẤT ĐẶC TRƯNG AI"

được sử dụng để khởi động toàn bộ quá trình xử lý tự động của hệ thống.

Sau khi được kích hoạt, chương trình sẽ:

1. Đọc dữ liệu PGN.
2. Gọi Engine Stockfish.
3. Trích xuất đặc trưng.
4. Đưa dữ liệu vào mô hình AI.
5. Trả về kết quả dự đoán.

Khung hiển thị kết quả

Khu vực này hiển thị:

- 14 đặc trưng đã được trích xuất.
 - Xác suất gian lận.
 - Kết luận cuối cùng của hệ thống.
-

5.2.2. Cơ chế diễn giải kết quả đầu ra

Thay vì chỉ đưa ra kết luận đơn thuần, hệ thống sử dụng xác suất dự đoán để phản ánh mức độ tin cậy của kết quả.

Trường hợp xác suất dưới 50%

Nếu:

Probability < 50%

Hệ thống gán nhãn:

✅ TRONG SẠCH (NORMAL PLAYER)

Điều này cho thấy chất lượng nước đi và các chỉ số kỹ thuật nằm trong phạm vi phù hợp với trình độ Elo của người chơi.

Trường hợp xác suất từ 50% trở lên

Nếu:

Probability $\geq$ 50%

Hệ thống gán nhãn:

⚠️ NGHI VẤN GIAN LẬN (CHEATER)

Mức xác suất càng cao thì độ tin cậy của nhận định càng lớn.

Ví dụ:

- 60%: Có dấu hiệu bất thường.
- 80%: Khả năng gian lận cao.
- Trên 90%: Có nhiều đặc điểm tương đồng với hành vi sử dụng Engine.

Việc kết hợp giữa dữ liệu đánh giá từ Stockfish và mô hình học máy giúp kết quả phân tích có tính khách quan và dễ giải thích hơn cho người dùng.

Hình 5.1 Demo giao diện hệ thống

Dữ liệu cheat

Tập dữ liệu sử dụng trong nghiên cứu được xây dựng từ bộ dữ liệu công khai và các mẫu mô phỏng hành vi gian lận bằng Engine Stockfish. Do đó, dữ liệu chưa phản ánh đầy đủ các hình thức gian lận phức tạp trong môi trường thi đấu thực tế, nơi người chơi có thể chỉ sử dụng Engine ở một số thời điểm quan trọng của ván đấu.

Tốc độ xử lý

Quá trình phân tích từng nước đi bằng Stockfish yêu cầu lượng tài nguyên tính toán đáng kể.

Thời gian xử lý trung bình hiện tại dao động từ 3 đến 5 giây cho mỗi ván cờ.

Trong tương lai, hệ thống có thể được tối ưu bằng:

- Kỹ thuật xử lý bất đồng bộ (Asynchronous Processing).
- Tối ưu hóa cấu hình Engine.
- Sử dụng các mô hình nén (Model Compression).

Bổ sung dữ liệu hành vi

Nghiên cứu hiện tại chỉ tập trung vào chất lượng nước đi.

Trong tương lai có thể mở rộng thêm các đặc trưng như:

- Thời gian suy nghĩ giữa các nước đi.
- Nhịp độ ra quyết định của người chơi.
- Hành vi sử dụng chuột và thao tác trên giao diện.
- Mẫu hình thay đổi phong độ trong suốt ván đấu.

Việc kết hợp thêm các nguồn dữ liệu hành vi sẽ giúp nâng cao độ chính xác của hệ thống và tăng khả năng phát hiện các hình thức gian lận tinh vi trong môi trường thi đấu trực tuyến.

TÀI LIỆU THAM KHẢO

[1] B. Badr and W. Feng, "Detecting Chess Cheating Thanks To ML & ANN Technology," *International Journal of Computational Engineering Research (IJCER)*, vol. 15, no. 2, pp. 1-8, Feb. 2025.

[2] H. Goldowsky, "How to catch a chess cheater: Ken Regan finds moves out of mind," *Chess Life*, pp. 18-30, Jun. 2014.

[3] K. W. Regan and G. M. Haworth, "Intrinsic chess analysis," in *Proceedings of the 23rd Benelux Conference on Artificial Intelligence (BNAIC 2011)*, 2011, pp. 248-255.

[4] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.

[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785-794.

[6] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T. Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," *Advances in Neural Information Processing Systems*, vol. 30, pp. 3146-3154, 2017.

[7] Lichess.org, *Lichess Open Database - Public Game PGN Logs*. [Online]. Available: <https://database.lichess.org>.

[8] B. Dandoy, "Spotting Cheaters: Chess cheating dataset," *Kaggle Dataset Repository*, 2024. [Online]. Available: <https://www.kaggle.com/datasets/brieucdandoy/chess-cheating-dataset/discussion?sort=hotness>.

[9] Chess.com, *Chess.com Fair Play Policy and Anti-Cheating Measures*. [Online]. Available: <https://www.chess.com/cheating>.

[10] T. Romstad, M. Costalba, J. Kiiski, and G. Linscott, *Stockfish: A chess evaluation engine via alpha-beta search and NNUE evaluation*, Open-source software repository. [Online]. Available: <https://stockfishchess.org/>.

[11] FIDE Fair Play Commission, *FIDE Anti-Cheating Regulations and Guidelines for Arbiters*, World Chess Federation, 2022.